Game Design Document

Alehandro Silvano Tanasa 240191 5S1M-B

✨ **MeteorShower**
: Minijuego espacial estilo arcade enfocado en supervivencia

Overview

Genre: Arcade / 2D Space Shooter

Target Audience: Jugadores de juegos retro

Experience: Disparar asteroides → Ganar puntos → Aumentar spawnrate → Sobrevivir

Platform: PC (Windows 64)

Unique Selling Points:

- Movimiento basado en físicas con masa
- Fragmentación de asteroides
- Reacciones en cadena
- Aleatoriedad en cada run

High-level Details

Main Objective:

Conseguir la mayor cantidad de puntos disparando asteroides sin colisionar con ellos.

Gameplay Pillars:

- Inercia y rotación de la nave
- Escalabilidad de la dificultad
- Rendimiento fluído
- Movimiento sin límite de velocidad
- Espacio infinito

Resources

▼ Concept / Ideas Board

- Futuras mejoras:
  - Power-ups
  - Leaderboard
  - Nuevos enemigos

▼ References

N/A

Gameplay

Core Gameplay & Game Mechanics

Controlar una nave en un entorno 2D para esquivar/destruir asteroides acumulando la máxima cantidad de puntos posible antes de colisionar.

- **Inercia física:** La nave se desplaza con un efecto de inercia y aceleración.
- **Disparo:** La nave dispara proyectiles que fragmentan y destruyen asteroides.
- **Fragmentación de asteroides:** Los asteroides se fragmentan en trozos más pequeños.
- **Dificultad:** La dificultad de la partida es escalable aumentando el spawnrate de los meteoritos.
- **Reacción en cadena:** La fragmentación de asteroides puede generar reacciones en cadena.

Controls

- **W / S / UP / DOWN** → Avanzar / Retroceder
- **A / D / LEFT / RIGHT** → Girar
- **ESPACIO** → Disparar
- **ESC** → Pausa

Objectives & Game Progressions

- **Short Term**: Destruir el asteroide o mini asteroide que amenace con acabar con la nave.
- **Mid Term:** Sobrevivir y ganar puntos destruyendo objetivos.
- **Long Term:** Jugar para conseguir un nuevo High-Score.
- **Progression:** El spawnrate de asteroides aumenta cada vez que un asteroide spawnea, haciendo la supervivencia cada vez más complicada. Sin embargo, hay un límite de asteroides para que la partida no se vuelva un caos.

Obstacles & Failure Conditions & Level Designs

Obstacles:

- Asteroides default
- Mini-asteroides

Failure Conditions:

- Colisionar directamente contra cualquier asteroide default o mini-asteroide Muestra la pantalla de “GAME OVER” con opción de empezar una nueva partida “NEW GAME”.

Level Design:

- Escena única con espacio infinito delimitada por la vista de la cámara.

Assets

Style Guidelines

- Estilo retro arcade 2D.

Art

Visual References / Inspiration

N/A

Palette / Moodboard

- Fondo oscuro negro, gran contraste con tonos naranjas y rojos para enemigos y disparos.

Concept Art

- Sprite de asteroide 2D con vista lateral.
- Sprite de asteroide 2D con vista lateral y escalado pequeño.

Character Designs

- Sprite de Nave 2D con vista superior.

Environment Aesthetics

- Fondo espacial estrellado.

Sound

N/A

Resources

- Librerías:
  - UnityEngine
  - UnityEngine.SceneManagement
  - UnityEngine.Pool
  - UnityEngine.InputSystem

Technical Requirements

Game Engine

- Unity (Versión 6000.6.2f1)

Language

- C#

Hardware Requirements

Minimum Specs

Mínimo para lo que consume Unity y un poco más:

- Procesador Dual Core 2.0 Ghz, 4GB RAM, grafica compatible con DirectX 11

Recommended Specs

- Procesador Quad Core, 8 GB RAM, Gtx 1050+

Required Features (Scripts y arquitectura)

SCRIPTS:

- **Player.cs:** Gestiona movimiento y rotación de la nave, inicializa la pool de bullets, gestiona los disparos, las condiciones de derrota y el espacio infinito.
- **Asteroid.cs:** Gestiona las colisiones entre asteroides, el comportamiento de mini-asteroides, su ciclo de vida y su spawn.
- **MiniAsteroids.cs:** Gestiona las colisiones entre dos mini-asteroides.
- **Bullet.cs:** Gestiona el movimiento de las balas, el retorno y salida de la pool y el score.
- **PauseMenu.cs:** Gestiona el funcionamiento del menu de pausa.
- **GameOverMenu.cs:** Gestiona el funcionamiento del menu de game over.
- **EnemySpawner.cs:** Gestiona el spawn de meteoritos, la dificultad de la partida y las colisiones y ciclo de vida de los asteroides.

ARQUITECTURA:

- **Bullet Pooling:** Se utiliza pooling para la optimización de rendimiento y spawn de bullets.
- **Ignore Collisions:** Se gestionan las colisiones de grandes cantidades de objetos en su spawn, no cuando la colisión ya se ha producido.